Seals in Motion.

Flexible Sealing Solutions for the Mobility of the Future.

Thesis Defense

Niederwinkling, 12.09.24

Wen-Hao Wu

Automated Visual Inspection of Sealing Products: Implementation and Evaluation of an AI-based System

Agenda

- Introduction
- Methodology
- Results
- Conclusion and Future Work

Agenda

- **Introduction**
- Methodology
- Results
- Conclusion and Future Work

Introduction

Background

- Wallstabe & Schneider manufactures different kinds of **sealing solutions**
- Quality Control in Wallstabe & Schneider consists of roughly **50% manual 50% automated**

Introduction

Problem Statement

- Most of the automatic methods utilize **traditional machine learning** methods, which have some **limitations**
- Proposed solution: Introduce **Deep Learning methods** for performance boost

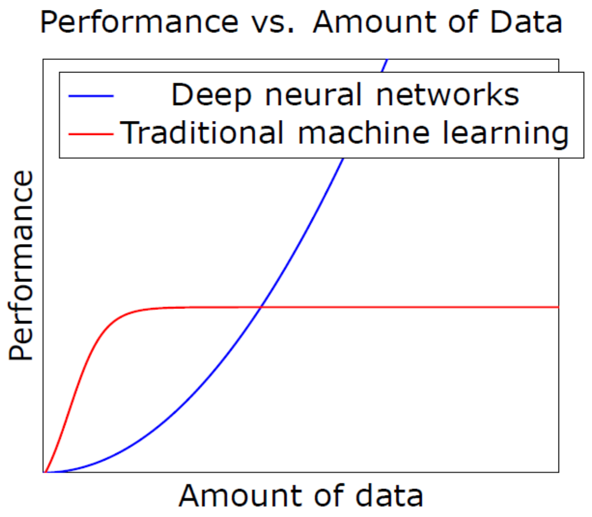


Figure 1.2: Performance comparison between traditional machine learning and deep neural networks as a function of the amount of data available [59]

Agenda

- Introduction
- **Methodology**
- Results
- Conclusion and Future Work

Data Collection: Selection of Sealing Products

- **Lip seals** are the study focus

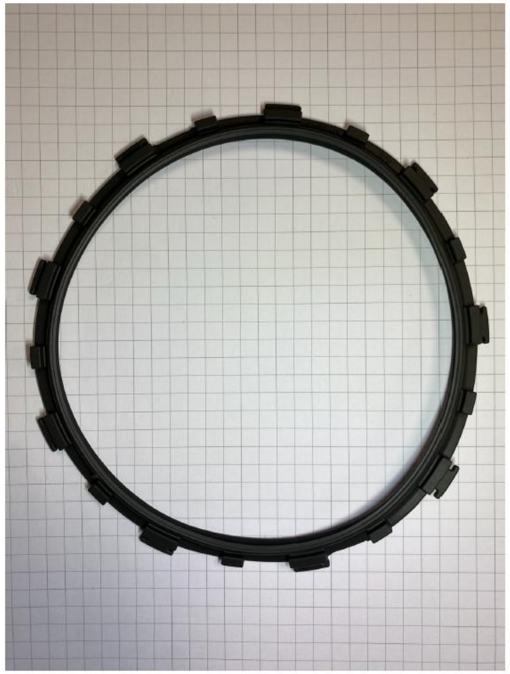


Figure 4.1: Lip seal from article 108.6, showing side A

Figure 4.2: Lip seal from article 108.6, showing side B

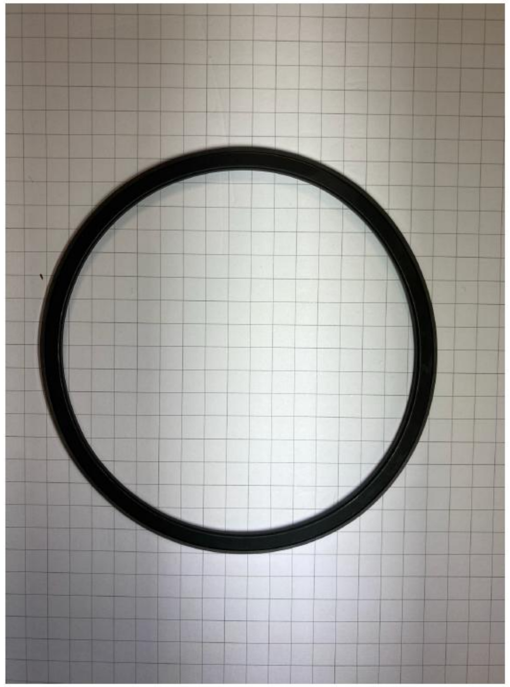


Figure 4.3: Lip seal from article 77.1, showing side A

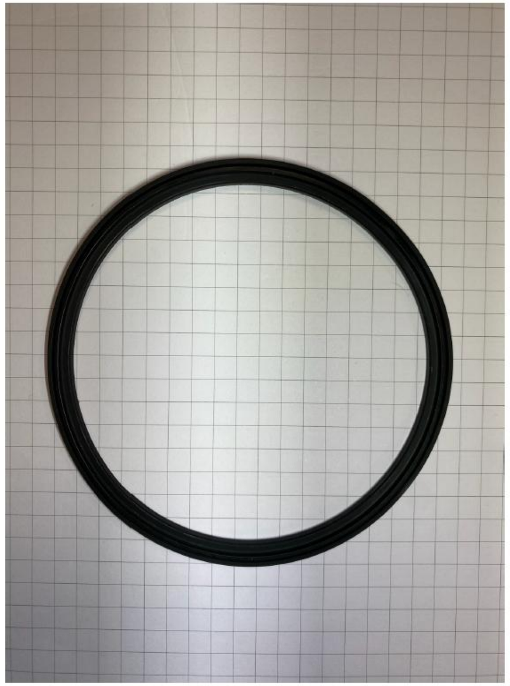


Figure 4.4: Lip seal from article 77.1, showing side B

Data Collection: Selection of Sealing Products

- **Lip seals** are the study focus
- Sealing articles are named after their **inner diameter**

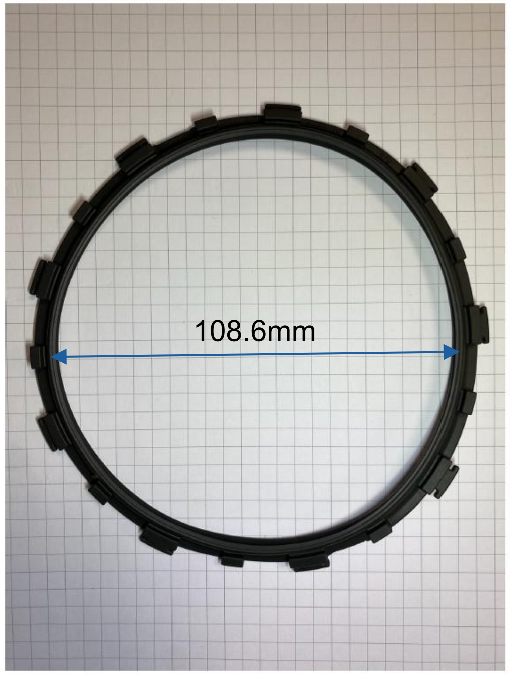


Figure 4.1: Lip seal from article 108.6, showing side A

Figure 4.2: Lip seal from article 108.6, showing side B

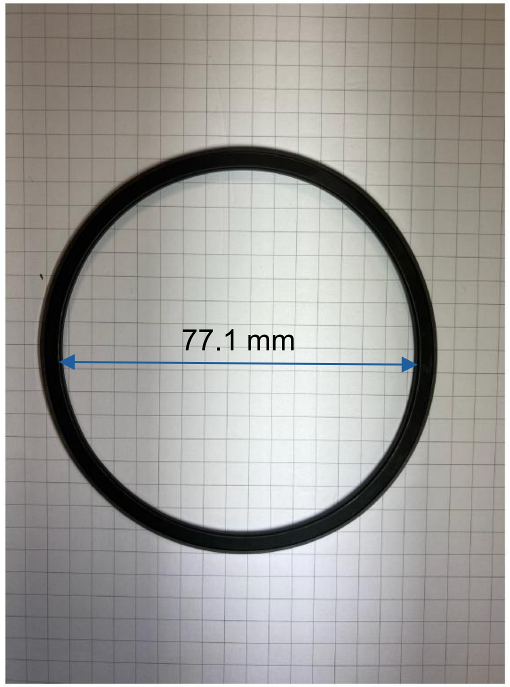


Figure 4.3: Lip seal from article 77.1, showing side A

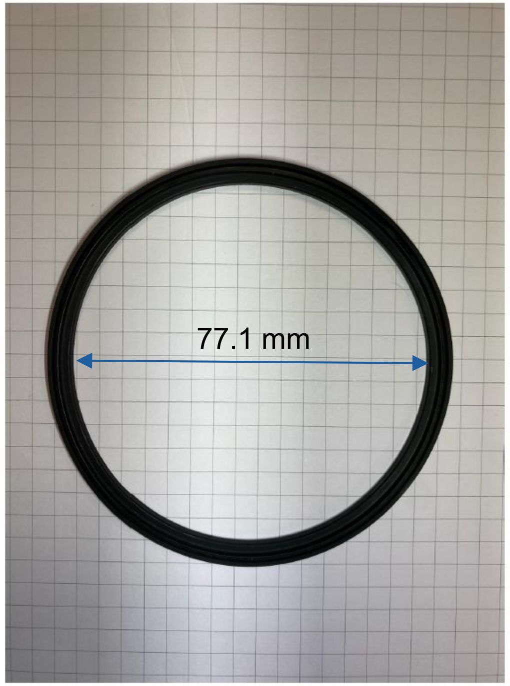
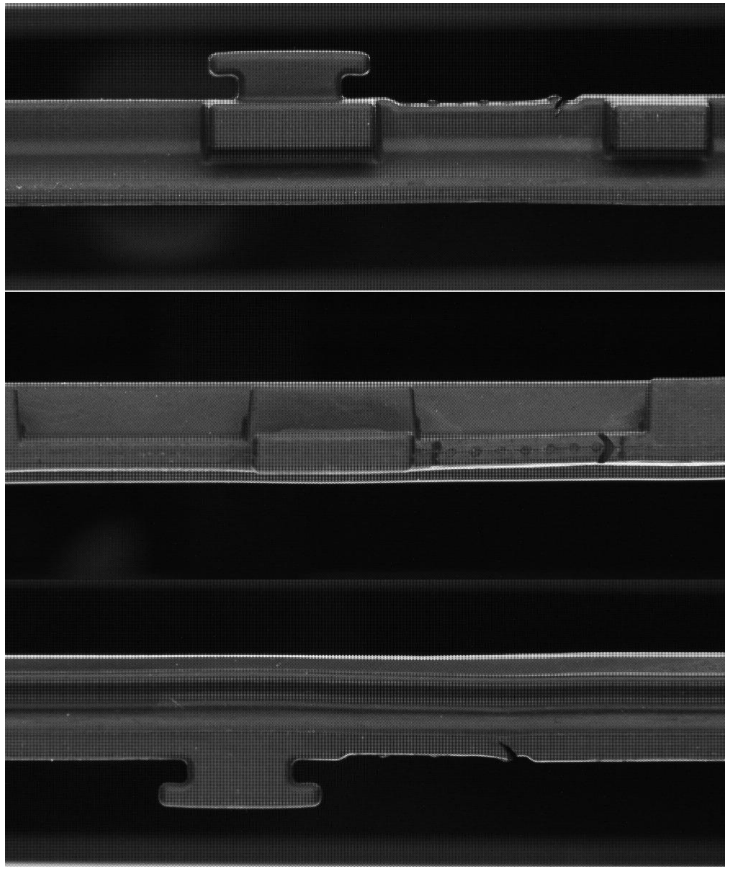


Figure 4.4: Lip seal from article 77.1, showing side B

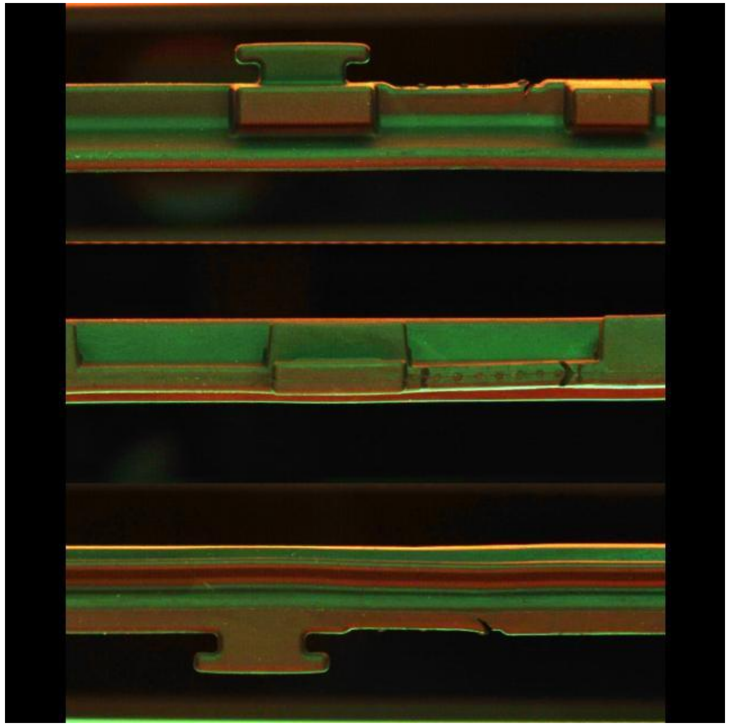
Data Collection: Image Acquisition

- Images captured from **inspection machine**
- Raw images are saved in **.pgm format** and **does not contain color information**



Data Collection: Image Preprocessing

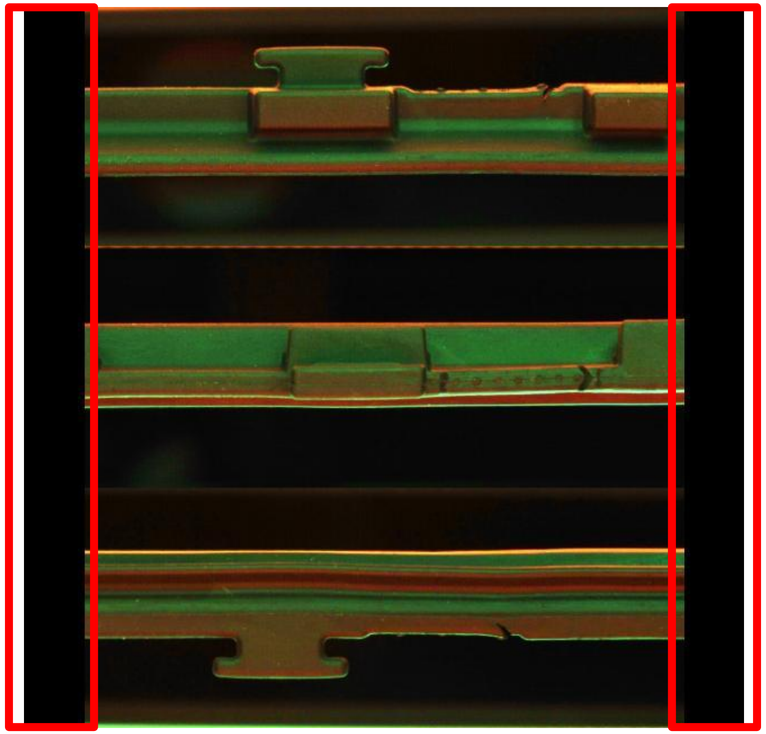
- Convert .pgm to **.jpg**
- Restore **color information**



Methodology

Data Collection: Image Preprocessing

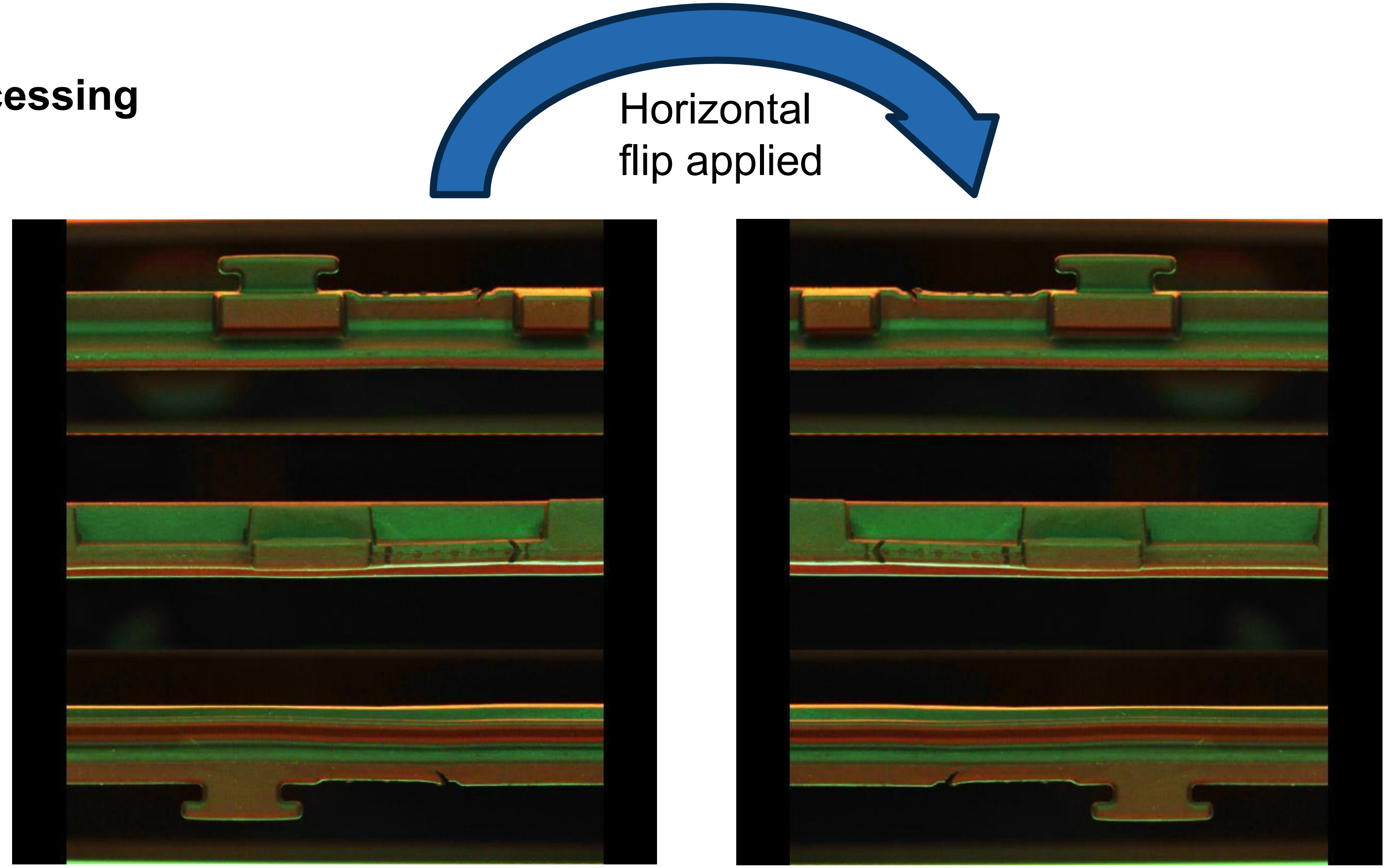
- Convert .pgm to **.jpg**
- Restore **color information**
- The sides are **filled with black**



Methodology

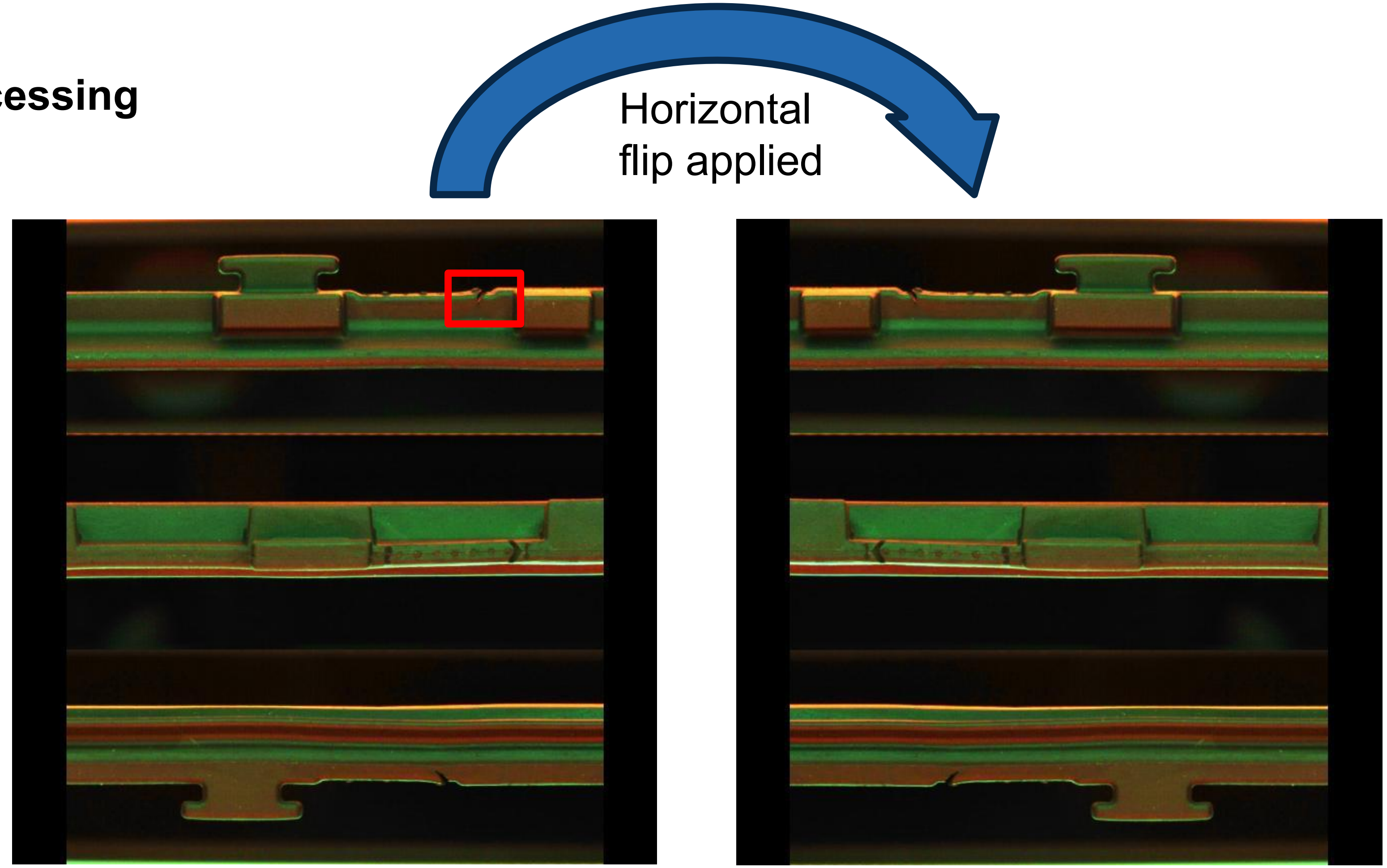
Data Collection: Image Preprocessing

- Convert .pgm to **.jpg**
- Restore **color information**
- The sides are **filled with black**
- **Augmentation** applied



Data Collection: Image Preprocessing

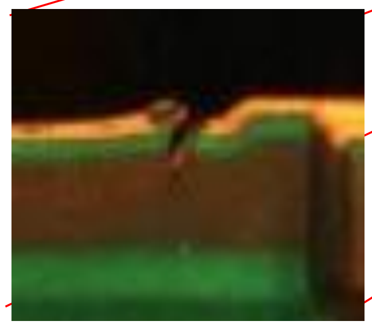
- Convert .pgm to **.jpg**
- Restore **color information**
- The sides are **filled with black**
- **Augmentation** applied

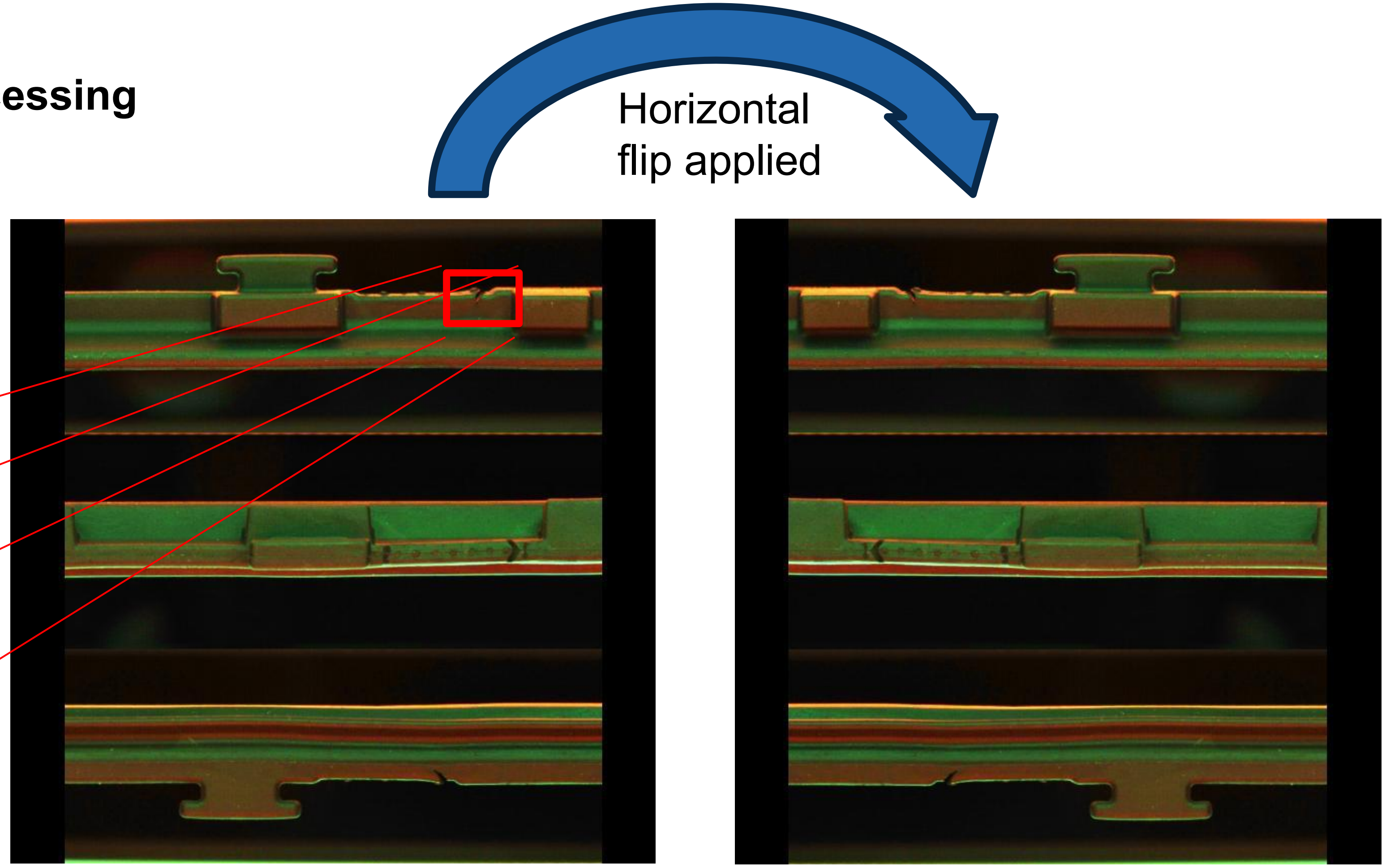


Methodology

Data Collection: Image Preprocessing

- Convert .pgm to **.jpg**
- Restore **color information**
- The sides are **filled with black**
- **Augmentation** applied





Data Collection: Defect Types

- Cut

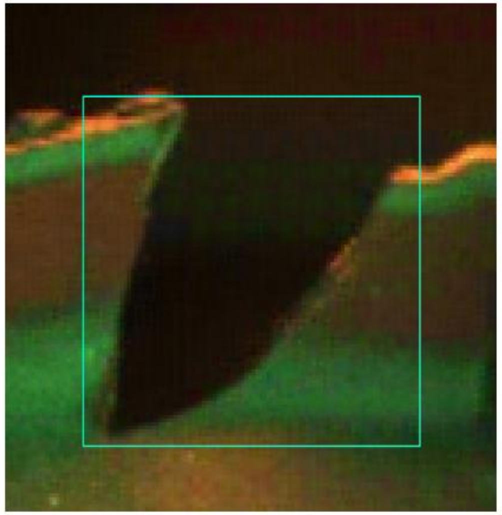


Figure 4.5: Close-up of cut, example 1

Data Collection: Defect Types

- Cut, Missing

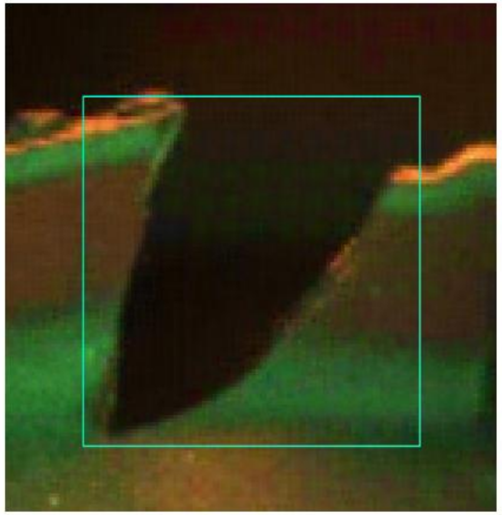


Figure 4.5: Close-up of cut, example 1

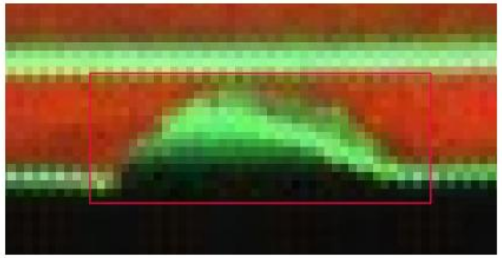


Figure 4.14: Close-up missing, example 4

Data Collection: Defect Types

- Cut, Missing, Rough Surface

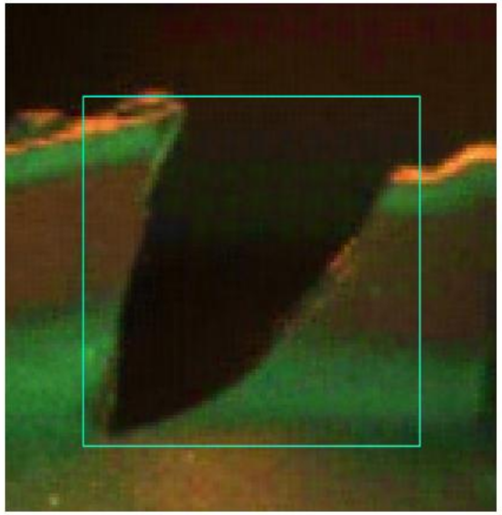


Figure 4.5: Close-up of cut, example 1

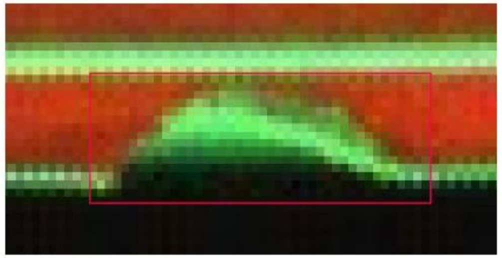


Figure 4.14: Close-up missing, example 4

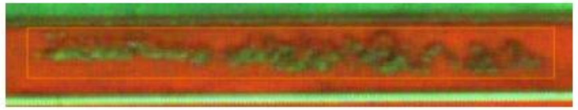


Figure 4.17: Close-up rough surface, example 1

Data Collection: Defect Types

- Cut, Missing, Rough Surface, Excessive Material

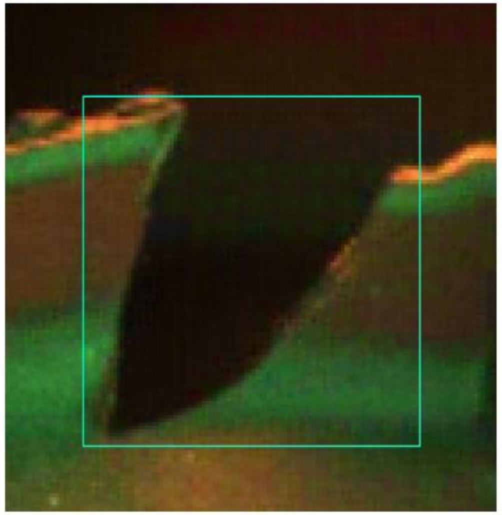


Figure 4.5: Close-up of cut, example 1

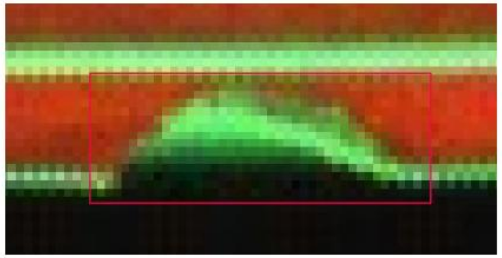


Figure 4.14: Close-up missing, example 4

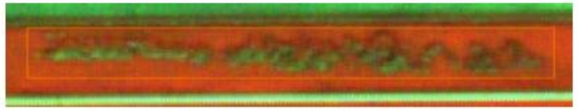


Figure 4.17: Close-up rough surface, example 1

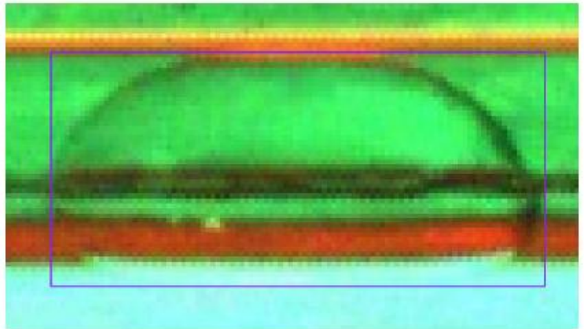
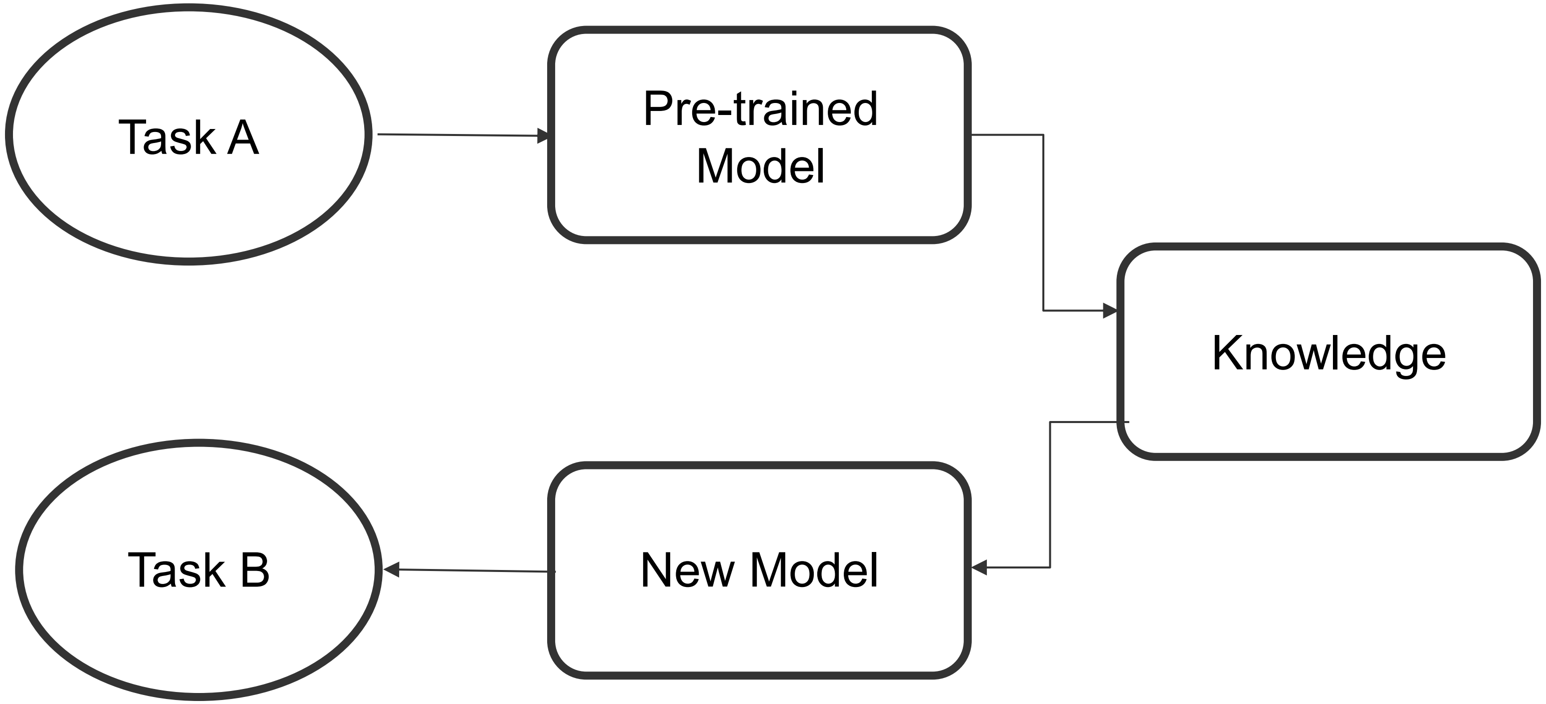


Figure 4.26: Close-up excess material, example 4

Model Training: Transfer Learning

- Main technique
- **Transfers knowledge** from one task to a related task
- Ideal for scenarios with **limited data** and **computational resources**
- **Reduced training time**



Model Training: Pre-trained Models

- **Tensorflow Object Detection (TFOD) Model Zoo**
  - Single Shot Detector (SSD) MobileNet V2 FPNLite 640x640
  - EfficientDet
- **You Only Look Once (YOLO)**
  - YOLOv8 family (n, s, m, l, x)
- Pre-trained on the COCO dataset

Methodology

Evaluation Metrics

- Mean average precision (mAP)
- Inference speed
- Training time
- (Ease of installation)

Model Export: Open Neural Network Exchange (ONNX)

- Open source project originally developed by Microsoft and Facebook
- Allows model conversion between different frameworks (e.g., PyTorch, TensorFlow).
- **Improves inference speed on CPU** by converting models to ONNX without sacrificing the accuracy.

Source: https://onnx.ai/

Agenda

- Introduction
- Methodology
- **Results**
- Conclusion and Future Work

Results

Preliminary Model Comparison and Selection

- **YOLOv8n out-performs SSD** in all metrics
- Performance of SSD was also reduced after introduced an additional class
- Further training attempts were conducted on all the models from the **YOLOv8 family**

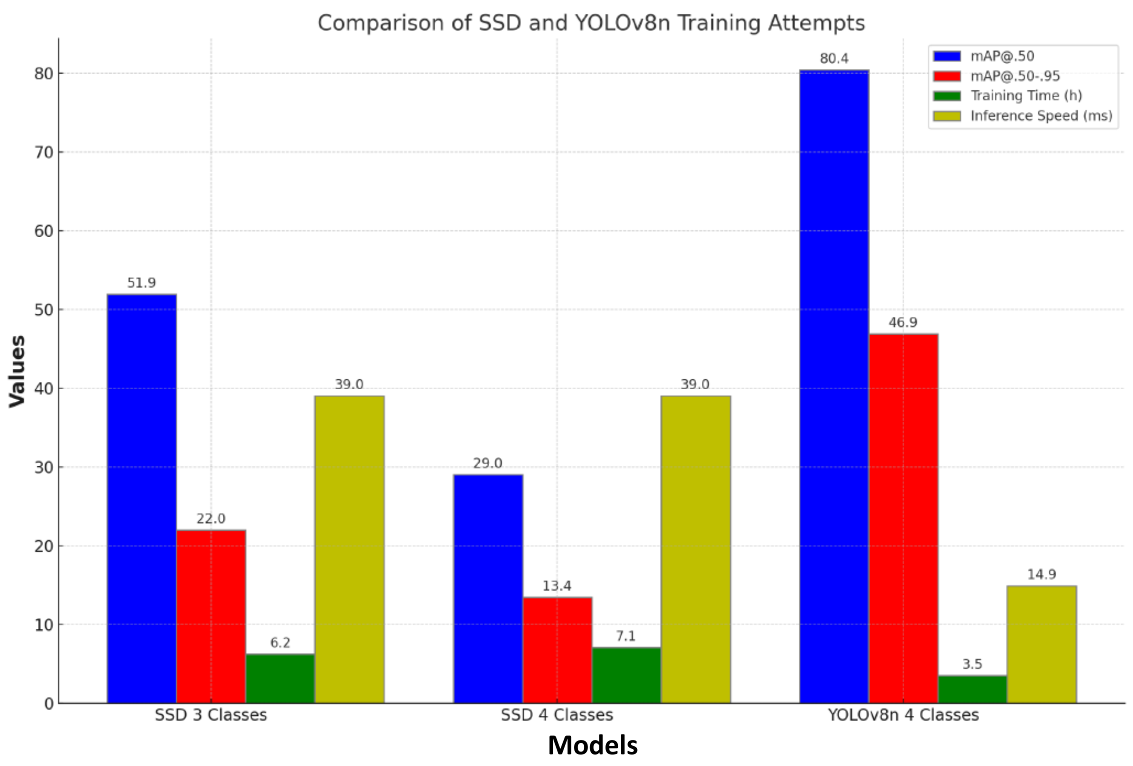


Figure 7.5: Comparison of SSD and YOLOv8n Training Attempts

Results

YOLOv8 Model Inference Time vs. mAP

- The choice of model depends on the **trade-off** between **speed** and **accuracy**
- The n model and the s model is more suitable for industrial environment

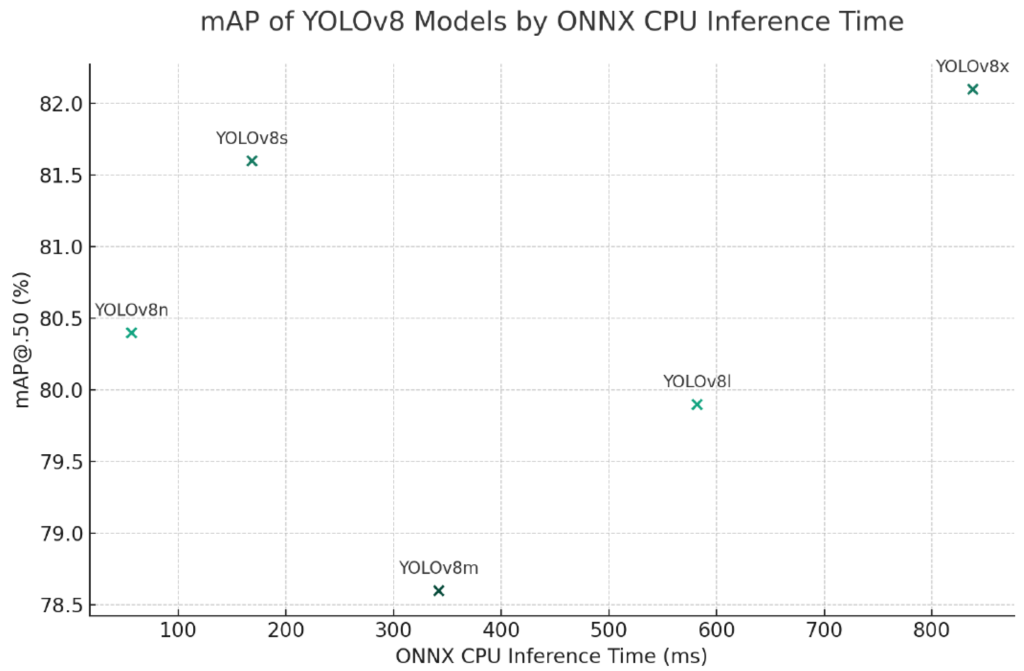


Figure 7.6: YOLOv8 model versions (n, s, m, l, x) based on their mAP@.50 scores plotted against their ONNX CPU inference times

Results

Inference Speed Comparison and Speedup Factors

- **2-2.5 times** inference speed from **.pt to .onnx on CPU**
- **4.5-9 times** inference speed from **CPU to GPU**
- **GPU** for **overall best** inference speed
- **.onnx** format for **best CPU** inference speed

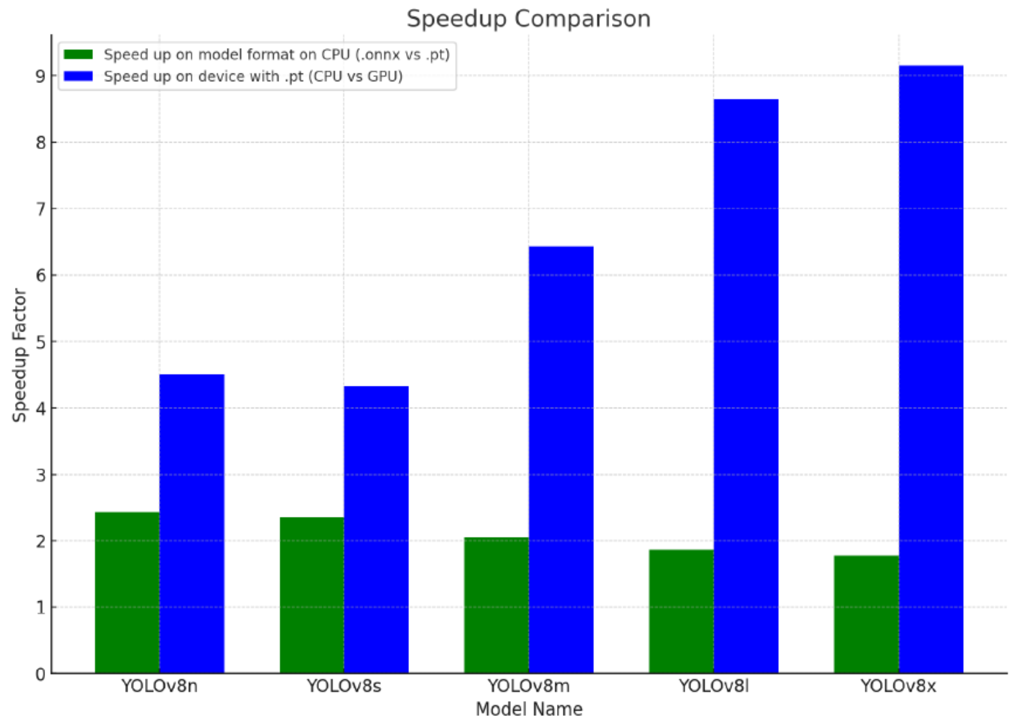


Figure 7.8: Speedup factors for YOLOv8 models: ONNX vs. PyTorch on CPU and CPU vs. GPU with PyTorch

Agenda

- Introduction
- Methodology
- Results
- **Conclusion and Future Work**

Conclusion and Future Work

Key Findings & Contributions

- **Fine-tuning YOLOv8 models** for surface defect inspection of sealing products
- Development of a user-friendly **GUI**
- Establishment and maintaining a **GitHub repository**
- **ONNX detection speed up** for CPU-based detection tasks

Conclusion and Future Work

Future Outlook

- **Expansion of datasets** to increase the ability to not only detect lip seals
- Increase **code base robustness**
- Continual enhancement of **dataset quality**
- **Deployment** of the proposed AI system onto **hardware** or **cloud platforms**

Thank you!